

Fast and Efficient Algorithm for Tibetan Jiu Chess Gameplay Data Collecting Based on Improved YOLOv8n Model

Xiali Li*, Yuanli Jia, Junzhi Yu*, and Feifan He

Abstract: Tibetan Jiu chess is a national intangible cultural heritage of People's Republic of China. The digital game platform has collected some online game data, but due to life habits, most masters and enthusiasts prefer to play offline. To collect offline gameplay data efficiently and quickly for Jiu chess, this paper proposes the intelligent algorithm based on improved You Only Look Once (YOLO) version 8 nano (YOLOv8n) model. Firstly, in order to improve the multi-scale feature fusion ability, the backbone and neck are replaced with co-designing and scaling ConvNets with masked autoencoders (namely ConvNeXt V2) and Gather-and-Distribute (GD) mechanism, respectively. Secondly, to reduce the parameter size of the optimized model, the Layer-Adaptive Magnitude-based Pruning (LAMP) score channel pruning algorithm is proposed. Finally, in order to convert the output of chess piece detection into standard chess manual data, the nearest Euclidean distance information matching algorithm and the game conversion algorithms are also proposed, separately. The experimental data source comes from the video recorded in 2023 National Tibetan Jiu Chess Competition in Qinghai Province. The experimental results show that the accuracy of our proposed model reaches 99.660% compared with other models, and the detection time after pruning is reduced by 5.5 ms. The results confirm that our algorithm can effectively improve and balance the speed and accuracy of chess piece detection.

Key words: computer games; object detection; Tibetan Jiu chess; You Only Look Once version 8 nano (YOLOv8n); Layer-Adaptive Magnitude-based Pruning (LAMP) score channel pruning

1 Introduction

Since Alpha Go victory over a human Go master in

- Xiali Li, Yuanli Jia, and Feifan He are with Key Laboratory of Ethnic Language Intelligent Analysis and Security Governance of Ministry of Education, School of Information and Engineering, Minzu University of China, Beijing 100081, China. E-mail: lixiali@muc.edu.cn; 23302130@muc.edu.cn; 23302088@muc.edu.cn.
- Junzhi Yu is with State Key Laboratory for Turbulence and Complex Systems, Department of Advanced Manufacturing and Robotics, College of Engineering, Peking University, Beijing 100871, China, and also with State Key Laboratory of General Artificial Intelligence, Beijing 100871, China. E-mail: junzhi.yu@ia.ac.cn.

* To whom correspondence should be addressed.

Manuscript received: 2024-09-14; revised: 2024-11-26;
accepted: 2024-12-10

2016^[1, 2], researchers have conducted a series of explorations on the application of deep learning in the field of computer games. In 2017, Google adopted a self-playing reinforcement learning approach to get rid of human chess players' knowledge guidance and use a simpler neural network for prediction and evaluation^[3]. The launch of AlphaGo Zero has taken mankind to a new level in the field of gambling game research. In 2017, Libratus^[4], an artificial intelligence developed by Carnegie Mellon University (USA), competed against four of humanity's top Texas Hold'em players for 20 days, and the Artificial Intelligence (AI) emerged victorious. In 2018, AlphaStar, a machine games program developed by DeepMind, defeated the pros with a 5:0 record. In July 2019, Pluribus, a collaboration between Facebook and Carnegie Mellon University, continued to evolve with self-gaming,

beating 15 top players in a six-player No Limit Hold'em^[5] event, marking another milestone in machine games. In 2022, the Tencent's AI Lab stunt team proposed Actor-Critic Hedge (ACH) under the framework of large-scale reinforcement learning algorithms^[6]. It partially solves the problem that large-scale deep reinforcement learning cannot converge from the game to the optimal solution of Nash equilibrium. So far, deep reinforcement learning has become a widely used method for high-level game AI, but existing models and algorithms require massive data resources or high-level computing power to support deep reinforcement learning training.

Tibetan Jiu chess is a national intangible cultural heritage of People's Republic of China. The size of the board is 14 grids $\times$ 14 grids. Tibetan Jiu chess game is divided into preparation and battle stages, each with unique and interdependent rules^[7]. The outcome of playing chess is greatly influenced by important moves, and the judgment is complex. Jiu chess lacks a large number of training datasets, and its self-playing deep reinforcement learning algorithm, such as hybrid deep reinforcement learning algorithm^[8], a phased game algorithm combining deep reinforcement learning and Upper Confidence bound apply to Tree (UCT) for Tibetan Jiu chess^[9], under the dual dilemma of computing power and data, the learning efficiency and decision-making accuracy of these algorithms are significantly restricted. Therefore, it is an important task to establish a shared chess manual dataset to improve the level of agents. Although the current Jiu chess game platform^[9, 10] can convert the game data of human chess players into game data, middle-aged and elderly people who are Tibetan Jiu chess enthusiasts and high-level chess players prefer offline games. Therefore, the scale and quality of the data collected by the digital platform are limited, which cannot meet the requirements of the large-scale training dataset required for the training of deep reinforcement learning models. The necessary way to improve the quality and scale of data collection is to study the intelligent collection of video image data of Tibetan Jiu chess offline games and automatically convert the images into standard chess manual data.

Chess piece detection is a key part of chess manual data collection. The category of chess pieces in Jiu chess is the same as that of Go, and the task of detecting chess pieces is to identify and classify black and white pieces. At present, researchers have carried

out a series of explorations on the detection of chess pieces in Go^[11, 12]. The quality of chess manual data directly affects the progress of Jiu chess research, so it is necessary to ensure the accuracy of the identified chess pieces in the process of chess piece detection. In order to prevent skipping a move of a chess player in the process of game conversion and affecting the accuracy of chess manual generation, high-speed detection is required. In addition, offline chess players cannot strictly land the chess pieces near the intersection point, so deviation from the chessboard grid line is common, and the existing Go chess piece detection methods cannot be effectively applied to the offline Jiu chess piece detection.

In recent years, AI algorithms^[13] represented by Deep Neural Networks (DNN), such as VGG^[14], ResNet^[15, 16], Fully Convolutional Networks (FCN)^[17], and DenseNet^[18], have been widely used in computer vision tasks, such as image recognition, object detection, semantic segmentation, and image generation, and have achieved remarkable results. You Only Look Once (YOLO) series is the single-stage object detectors based on deep neural networks, which is effective in balancing speed and accuracy, and can quickly and reliably identify objects in images^[19]. Moreover, it has low hardware requirements and has been widely used in robots, driverless cars, and video surveillance. Ultralytics released YOLOv8 in January 2023 is faster and more accurate in detection. Therefore, we propose an intelligent algorithm for the collection of offline chess gameplay data of Tibetan Jiu chess based on the improved YOLOv8n model, so as to complete the detection of Tibetan Jiu chess pieces with high precision and speed with low hardware resource consumption, and then convert the results of object detection into standard Jiu chess manual. The overall process of the intelligent algorithm is divided into chessboard extraction, chess piece recognition, and game conversion, as shown in Fig. 1. Chessboard extraction converts the original image into a standard image output as input for chess piece recognition. Chess pieces recognition is carried out by the optimized YOLOv8n model and the proposed Layer-Adaptive Magnitude-based Pruning (LAMP) score channel pruning algorithm, and the nearest Euclidean distance information matching algorithm is used to complete the information matching, and the output is an array of 14 rows and 14 columns. In the game conversion stage, the game conversion algorithms in

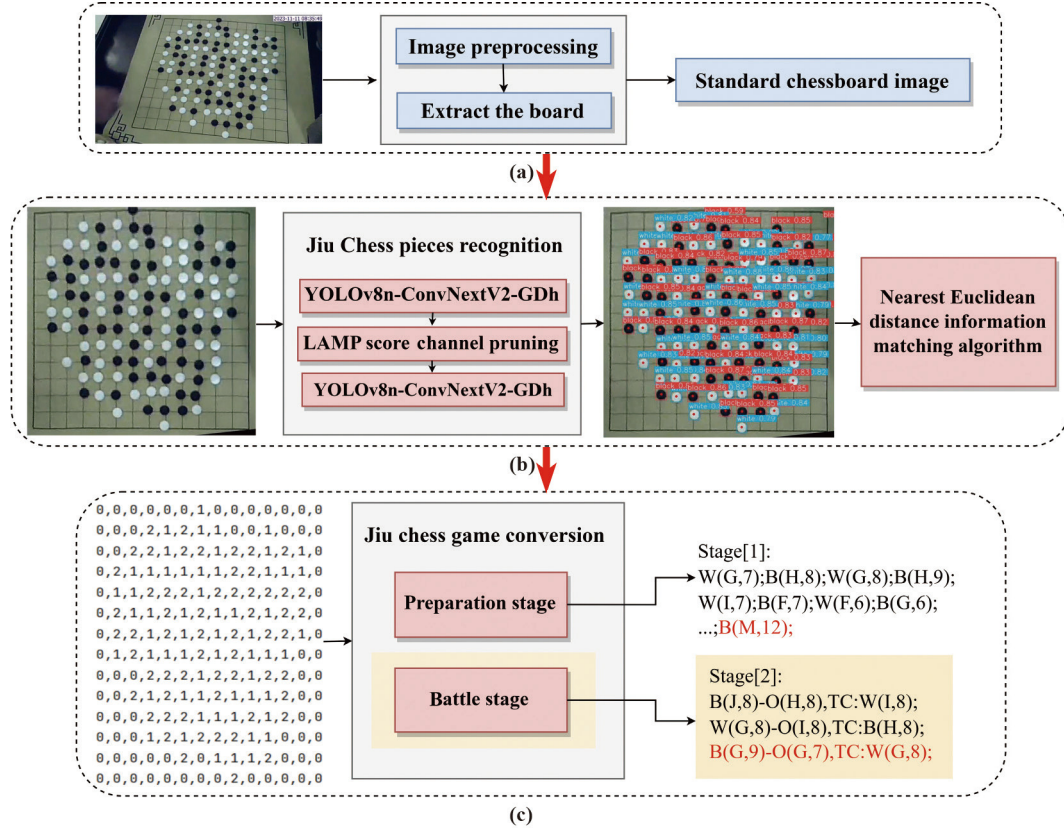


Fig. 1 Overall process of offline game data collecting algorithm, including three phases chessboard extraction, chess pieces recognition, and chess game conversion.

the preparation and battle stages are used to convert the array of 14 rows and 14 columns of information matching output into a standard Smart Game Format (SGF) game file. Figure 1 shows the blacks in the preparation phase, and the red font in the dotted box indicates the output of the image at the current frame.

The pink boxes in Fig. 1 are the innovation point of this paper, which are described as follows.

- The YOLOv8n-ConvNeXtV2-GDh is proposed to detect chess pieces efficiently and quickly by improving YOLOv8n model. Co-designing and scaling ConvNets with masked autoencoders (namely ConvNeXt V2) is used as the backbone network, the Gather-and-Distribute (GD) mechanism of adaptive YOLOn is used to optimize the neck structure, cut a conv block from all detect module. The chess piece detection accuracy of YOLOv8n-ConvNeXtV2-GDh reaches 99.660%, and the chess piece detection time is 13.8 ms. It effectively improves and balances the speed and accuracy of chess piece detection.

- The LAMP score channel pruning algorithm is proposed, the LAMP score is used to calculate the pruning weights of the channels. The LAMP score is

used to calculate the weights for channel pruning of YOLOv8n-ConvNeXtV2-GDh. After pruning, the detection accuracy is 99.629%, but the speed is reduced to 8.3 ms, which further improved the detection speed of the model.

- The nearest Euclidean distance information matching algorithm is proposed, and the array of 14 rows and 14 columns output by the nearest Euclidean distance information matching algorithm is used as the input data of the game conversion algorithm. Then, the position of the nearest chessboard intersection is searched according to the position of the center point returned by the chess piece detection and the position is determined to be the position of the falling piece. In order to obtain the standard Jiu chess manual data, Jiu chess game conversion of preparation stage algorithm and Jiu chess game conversion of battle stage algorithm are proposed.

2 Related Work

2.1 Status of digital game platform

Appropriate datasets and sufficient computing

resources are prerequisites for conducting research, so far, the game platform has 2417 users, 2839 games have been played with 2323 games of effective chess manual data. However, most competitions of Jiu chess are held offline, and higher-level chess players prefer to participate in offline competitions. According to the actual task requirements, this paper proposes an intelligent collection algorithm for Tibetan Jiu chess offline manual data based on the improved YOLOv8n model to complete the intelligent collection task of Jiu chess.

2.2 YOLOv8n and its improvements

Object detection is one of the most challenging problems in computer vision^[20]. Deep learning based object detection has evolved into two-stage detectors and one-stage detectors. Since the Regionbased Convolutional Neural Networks (RCNN)^[21, 22], which is regarded as reborn from Convolutional Neural Network (CNN)^[23], was proposed, one-stage detectors can infer all target objects in the detection image in a single step, making it easy to deploy and capable of real-time detection.

YOLO, as the first one-stage detectors^[24], was proposed in 2015. Subsequent versions of YOLO have been proposed after that. YOLOv8 is anchor-free, its speed and accuracy of detection have been further improved. The network structure of YOLOv8n consists of four layers: input, backbone, neck, and head.

There are two main ways to improve the YOLOv8n network framework to better adapt to the needs of downstream tasks. One is to meet the requirements of detection accuracy in practical engineering, researchers superimpose convolution modules, introduce attention modules, or replace some modules in the network structure, such as YOLOv8n-PD^[25] and RCM-YOLO^[26]. This improved method can improve the feature extraction ability, but the training speed is slowed down due to the increase of parameters. The other improved way is to meet the detection speed requirements, researchers are more committed to lightening the network structure of YOLOv8n, such as YOLOv8n-FastNet^[27], YOLOv8n-MobileNet^[28], YOLOv8n-vegetable^[29], and YDHNet^[30]. Although these methods improve the detection speed, the accuracy is reduced.

For the task of Tibetan Jiu chess real-time piece recognition, accuracy and speed are both important. The current improvements of YOLOv8n are not

suitable for Tibetan Jiu chess obviously. This paper proposed YOLOv8n-ConvNextV2-GDh model based on YOLOv8n, which can collect Tibetan Jiu chess game play data with high-speed and high-precision.

3 Method

The algorithm is divided into three phases, namely chessboard extraction, chess piece recognition, and game conversion, as shown in Fig. 1. Due to chessboard extraction uses a more common method, we will not go into detail here, but focus on the chess piece recognition and game conversion.

3.1 YOLOv8n-ConvNextV2-GDh

In order to solve the problems of inaccurate chess piece detection, excessive model parameters, and excessive model size in the traditional network, this paper proposes a new chess piece detection network model called YOLOv8n-ConvNeXtV2-GDh, as shown in Fig. 2, which is based on the YOLOv8n model, improves and optimizes it from three parts: backbone, neck, and head. Compared to Fast RCNN, as the one-stage detectors, YOLO series have a high detection speed, which can infer all target objects in the detection image in one step and deploy easily. YOLOv8 is anchor-free, the number of prediction boxes has been significantly reduced compared to previous versions, such as YOLOv5 and YOLOv7. Its speed and accuracy of detection have been further improved. YOLOv10 model eliminates the need for Non-Maximum Suppression (NMS) by introducing a two-layer allocation strategy, resulting in faster and more efficient object detection. In the late stage of the preparation stage and the early stage of the battle stage, the number of chess pieces on the board is large and densely distributed, and a large number of prediction boxes will interfere with the recognition. In order to ensure the accuracy of the collected games, a good balance of speed and precision is required, so YOLOv8 is suitable as the basic framework of the chess piece detection algorithm of Jiu chess. YOLOv8n is the smallest network in the YOLOv8 series models. Therefore, YOLOv8n is selected as the base model to improve.

Firstly, the ConvNext V2 is introduced into the backbone to enhance the ability of global information feature extraction and improve the detection accuracy of chess pieces. Secondly, the GD mechanism is introduced to optimize the neck structure to improve

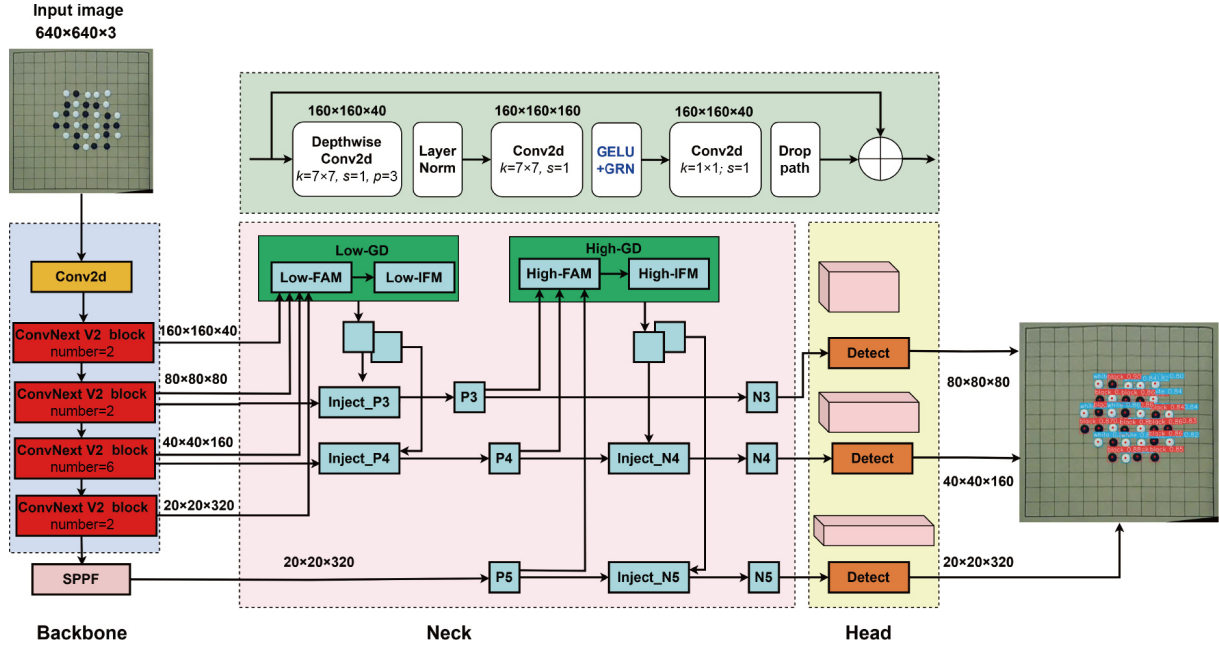


Fig. 2 Model structure of YOLOv8n-ConvNextV2-GDh.

the fusion ability of multi-scale information and further improve the accuracy of detection. In addition, the structure of the detection has been adjusted to improve the detection speed of chess pieces.

3.1.1 ConvNeXt V2

The Jiu chess game has the characteristics of multiple objectives and small objectives, and it requires a high accuracy of chess piece detection. The global information feature extraction ability of the backbone in YOLOv8n needs to be improved, which cannot well meet the requirements of detection accuracy. Therefore, in this paper, ConvNeXt V2^[31] is introduced into the network, as shown in the blue shaded area in Fig. 2, which can realize self-supervised learning, enhance the ability of global information feature extraction, and improve the detection accuracy. If the input feature image has $160 \times 160 \times 40$ pixels, the light green shaded area in Fig. 2 shows a network structure of the ConvNeXt V2 block, where k is the kernel size and s is the size of slide, p is the padding. In addition, the network structure of the backbone is adjusted accordingly to complete the feature extraction of the Jiu chess images.

The input images are the same size as those in the original YOLOv8n network, with a size of $640 \times 640 \times 3$ pixels. Firstly, the input images are subjected to a convolution operation with the slide of 4 and the Conv kernel of 4, and normalization. Then the output images are passed through 2 connected ConvNeXt V2 block

modules. All the red boxes in the blue shaded area in Fig. 2 represent the connected ConvNeXt V2 block modules. Then output the images of $160 \times 160 \times 40$ pixels. The downsampling operation is performed and the processed image are input into six connected ConvNeXt V2 block modules, and the $40 \times 40 \times 160$ pixels feature extraction images are output. Then, 2 connected ConvNeXt V2 block modules are fed in and the images with $20 \times 20 \times 320$ pixels are output. Finally, pooling and linear downsampling are passed into the Spatial Pyramid Pooling Fast (SPPF) network structure.

3.1.2 GD mechanism of adaptive YOLOn

The neck structure of YOLOv8 uses the traditional Feature Pyramid Network (FPN) structure, which contains multiple branches for multi-scale feature fusion, but it can only fuse features from adjacent layers, and the information of other layers can only be obtained indirectly through recursion. In order to prevent the loss of global feature information extracted from the backbone and improve the ability of multi-scale feature fusion between adjacent layers and cross-layers, the GD mechanism proposed by Wang et al.^[32] is introduced to achieve a relative balance between the speed and accuracy of Jiu chess piece detection, as shown in the pink shaded area in Fig. 2. On this basis, the input image size of the whole mechanism module is adjusted accordingly for the output of backbone, and it is applied to the YOLOn structure for the first time.

GD mechanism consists of two parts: Low-stage Gather-and-Distribute branch (Low-GD) and High stage Gather-and-Distribute branch (High-GD), and the network structure is shown in Fig. 3. The two branches can enhance the multi-scale feature fusion ability and the model's ability to detect objects of different sizes, and improve the accuracy of model detection. The Low-GD is shown in Fig. 3a. Firstly, the features B2, B3, and B4 output by all ConvNeXt V2 block modules in the backbone and feature B5 output by SPPF module are fused to obtain high-resolution features that retain small target information. Then, all these features are passed into Low-stage Feature Alignment Module (Low-FAM), and the Low-level Information Fusion Module (Low-IFM) and information injection (Inject_P4 and Inject_P3) are carried out in the Low-GD, and the corresponding features P3, P4, and P5 are output. The low-level feature alignment module adopts operations, such as average pooling to unify the sizes of all feature images. The low-level information fusion module includes Reparameterized Convolutional blocks (RepConvblock) and split. The high-GD is shown in Fig. 3b. The features P3, P4, and P5 output from the Low-GD are passed into the High Feature Alignment Module (High-FAM), the High-level Information Fusion Module (High-IFM) and information injection (Inject_N4 and Inject_N5) are carried out again in the High-GD branch. The biggest difference from Low-GD is the addition of a multi-head attention mechanism and a front feedback neural network in High-IFM. Finally, the High-GD outputs are passed into the detection head.

3.1.3 Improved detect module

To further improve the detection speed, cut out a Conv

for each detect branch in head. The detect module in YOLOv8n consists of two branches, one branch passes through two Convs, a Conv2d, and Bounding box regression Loss (BboxLoss), and the other branch passes through two Convs, a Conv2ds and Classification Loss (ClsLoss). Although the structure ensures the high accuracy of the detection results, the addition of two Convs increases the number of parameters of the model and slows down the generation of detection results. Therefore, to reduce the parameters of the model and improve the overall detection speed of the model, for the two branches of the detect in YOLOv8n, this paper cuts out a Conv in each branch and retains only a Conv, as shown in Fig. 4.

3.2 LAMP score channel pruning

Compared with the YOLOv8n model, the YOLOv8n-ConvNeXtV2-GDh model has a significant improvement in the detection effect of chess pieces, but its number of parameters is very large, which is a great challenge to hardware resources and detection speed. The existing channel pruning algorithm^[33] can effectively reduce the size of model parameters, but it prunes the network channels with small absolute values that contribute to the generalization performance of the model, resulting in the distortion of the model. Therefore, this paper proposes an LAMP score channel pruning method. In this method, the network contribution parameters are calculated based on the amplitude, which prevents the model distortion caused by pruning, and greatly reduces the number of parameters and calculations of the model while reducing the accuracy loss as much as possible.

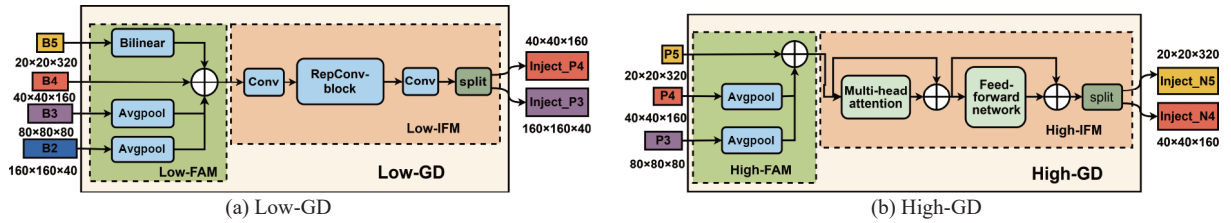


Fig. 3 Structure of GD mechanism of adaptive YOLOn.

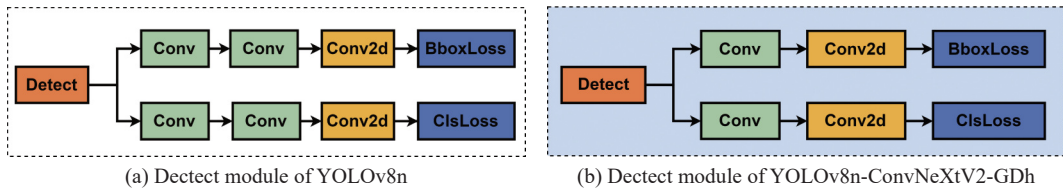


Fig. 4 Detect module comparison between YOLOv8n and YOLOv8n-ConvNeXtV2-GDh.

The principle of LAMP score is shown in Fig. 5. The network contribution parameter is calculated by the amplitude-based pruning method LAMP score^[34], and then the channel with the smallest network parameter contribution is deleted and the other channels are retained. The channel-level sparsity of the convolutional layer is enhanced by applying L_1 regularization on the channel scaling factor, and the specific formula of regularization is as follows:

$$L_1 = \sum_{(x,y)} l(f(x, \omega), y) + \lambda \sum_{\gamma \in \Gamma} g(\gamma) \quad (1)$$

where x and y represent the original image of Jiu chess and the output image of the chess piece object detection, respectively; $l(f(x, \omega), y)$ is the loss function, which is used to measure the error between the prediction of the model and the true label, $f(x, \omega)$ is the prediction function of the model, ω represents the trainable weight; $g(\cdot)$ is the penalty of the scaling factor caused by sparsity; γ is the certain parameters of the model, such as weights, biases, or other specific combinations of parameters; Γ is the scaling factor of the batch normalization layer; and λ is the balance parameter.

The LAMP score is calculated as

$$\left[\begin{array}{c} \frac{u_1^2}{u_1^2} \\ \frac{u_1^2}{u_1^2 + u_2^2} \\ \frac{u_1^2}{u_1^2 + u_2^2 + u_3^2} \end{array} \right] \left\| \left[\begin{array}{c} \frac{v_1^2}{v_1^2} \\ \frac{v_1^2}{v_1^2 + v_2^2} \\ \frac{v_1^2}{v_1^2 + v_2^2 + v_3^2} \end{array} \right] \right. \quad (2)$$

where u_1, u_2, u_3 and v_1, v_2, v_3 are the weights of different channel layers. The LAMP score performs layer-wise magnitude-based pruning by computing the squared magnitudes of all scaling factors in the current

layer, applying global average pooling to aggregate channel importance, and automatically determining layer-specific sparsity thresholds for channel removal.

3.3 Nearest Euclidean distance information matching algorithm

The pruning YOLOv8n-ConvNextV2-GDh model is used to detect chess pieces, and the output result is the category of the detected chess pieces and the coordinate position information of the center point of the chess pieces in the current image. At this time, the standard Jiu chess game cannot be obtained, and the output result needs to be further processed to achieve the task of game conversion. When playing offline, it is common for the position of the pieces to deviate from the intersection. In different Jiu chess games, the size of the chess pieces is different, and the relationship between the diameter of the chess pieces d and the side length D of the checkerboard grid is satisfied $d \leq D$, as is shown in Fig. 6a. In practice, the chess piece may be offset in a certain direction, assuming that the direction of the offset is in the direction of the red line, and the position range of the chess piece offset is shown in the red line, and the center point of the chess piece is recorded as the position of the chess piece.

In order to solve the above problems, this paper proposes a nearest Euclidean distance information matching algorithm, which inputs the results of chess piece detection and outputs an array of 14 rows and 14 columns, as shown in Fig. 1c. The plane coordinates of the four points of the detection frame are (x_1, y_1) , (x_2, y_1) , (x_2, y_2) , and (x_1, y_2) , clockwise from the upper left corner, the intersection of the checkerboard grid is (X, Y) . The core formula of the algorithm is as follows:

$$\sqrt{((y_1 + y_2)/2 - Y)^2 + ((x_1 + x_2)/2 - X)^2} < \frac{D}{2} \quad (3)$$

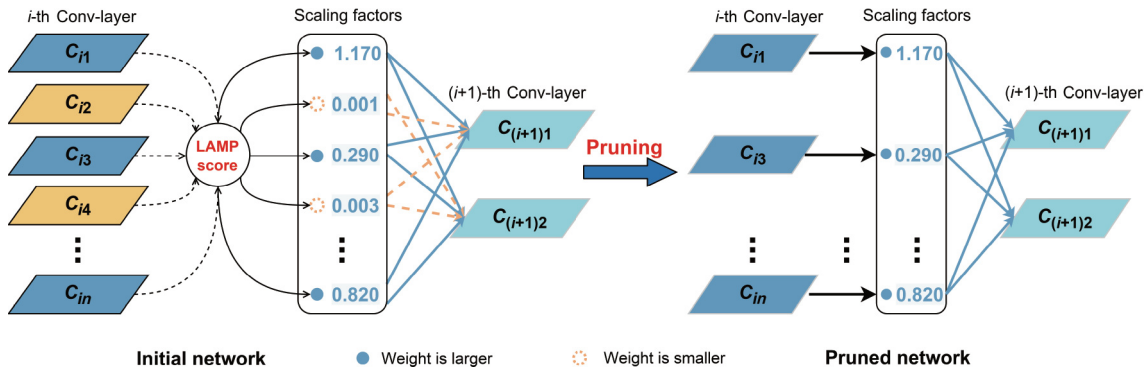


Fig. 5 LAMP score channel pruning ($C_{i1}, C_{i2}, \dots, C_{in}$ denote the input channels, $C_{(i+1)1}$ and $C_{(i+1)2}$ are the output channel).

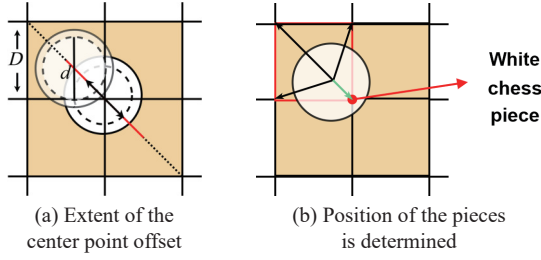


Fig. 6 Principle of chess piece deviation and information matching algorithm.

If the Euclidean distance between the center point of the piece and the intersection of a checkerboard around it is less than half the length of the checkerboard, then the category of the piece at the intersection is marked as the category of object detection of the piece. As shown in Fig. 6b, only the green four arrows are less than half of the length of the checkerboard grid, so the category at the red dot is marked as 2 on the array of 14 rows and 14 columns grids, that is, the identified piece is a white piece, which is described in Algorithm 1 for details.

3.4 Jiu Chess game conversion algorithm

After the information matching is completed, the output data are a chessboard array of the current frame

Algorithm 1 Nearest Euclidean distance information matching

Require: $A = a_1, a_2, \dots, a_\beta, \dots, a_{\text{num_p}}, a_\beta (1 \leq \beta \leq \text{num_p})$ stores coordinates of the four corner points of the bounding box for each detected chess piece, along with its corresponding category of the piece (0: empty chess piece, 1: black chess piece, and 2: white chess piece); num_p is the number of pieces; $C = c_1, c_2, \dots, c_\theta, \dots, c_{\text{num_i}}, c_\theta (1 \leq \theta \leq \text{num_i})$ stores the coordinates of each checkerboard intersection, num_i is the number of intersections

Ensure: A 14×14 two-dimensional array (as shown in Fig. 1c, upper-right corner) denoted by a matrix $\text{Phase} [14, 14]$, whose elements comprise exclusively the integers 0, 1, and 2

```

1: Set  $\text{Phase}[14, 14] = 0$ ,  $\text{row} = 0$ ,  $\text{col} = 0$ ;
2: for  $\theta = 1$ ;  $\theta < \text{num\_i}$ ;  $\theta ++$  do
3:    $\text{row} = \text{int}(\text{round}(c_\theta[0]/D))$ ;
4:    $\text{col} = \text{int}(\text{round}(c_\theta[1]/D))$ ;
5:   for  $\beta = 1$ ;  $\beta < \text{num\_p}$ ;  $\beta ++$  do
6:     if  $\sqrt{((a_\beta[0] + a_\beta[2])/2 - c_\theta[1])^2 + ((a_\beta[1] + a_\beta[3])/2 - c_\theta[0])^2} < D/2$  then
7:        $\text{Phase}[\text{row}, \text{col}] = a_\beta[4]$ ;
8:     end if
9:   end for
10: return  $\text{Phase}[\text{row}, \text{col}]$ ;
11: end for
```

image, that is, a two-dimensional array of 14×14 with values of 0, 1, and 2, representing empty pieces, black pieces, and white pieces, respectively. In order to further obtain the standardized manual data of Jiu chess, it is necessary to design the preparation and battle two-stage game conversion algorithm according to the manual data specification shown in Table 1.

3.4.1 Jiu chess game conversion of preparation stage

The basic principle of the algorithm in the preparation stage is to compare the value of the current array Phase_now with the output Phase_last of the previous frame image information matching, that is, whether 0 becomes 1 or 2. For example, if the value of the array changes from 0 to 1 to indicate a move by a black player, the string of the recorded game Stage_first added to the game is added to the game of black chess, and the record_step value of the number of moves is increased by 1, as described in Algorithm 2.

3.4.2 Jiu chess game conversion of battle stage

Suppose it is currently a black move, set record_step (the number of moves) to determine which side moves, the value of the record_step is 196 after all the games in the preparation stage are converted, and the record_step is 198 after the middle two pieces are removed, and the game conversion of battle stage algorithm is started until the preparation stage is completed. The basic principle of the algorithm is as follows. Firstly, the captured chess pieces are found, and the captured chess pieces are recorded with the intermediate variable string, which contains the category of the chess pieces and the position of the chess pieces, and the count stores the number of captured chess pieces. Then we look for the initial and

Table 1 Game data specification.

Character	Interpretation
"Stage [1]"	Indicates the entry into the battle stage
"Stage [2]"	Indicates the coordinates of the white chess
"W (x, x)"	Indicates the coordinates of the white chess
"B (x, x)"	Indicates the coordinates of the black chess
"O (x, x)"	Indicates the vacant coordinates
"TC:"	Indicates skipping to eat
"FC:"	It means eating seeds in a square
"QC:"	It means it is a gun array
"W (x, x)-O (y, y):"	Indicates that white moves from the point (x, x) to the vacant point (y, y)
","	Indicates the separator between black and white

Algorithm 2 Jiu chess game conversion of preparation stage

Require: Phase_last: output of the previous frame image information match; Phase_now: output of the image information match for the current frame; record_step: number of moves in the current move

Ensure Stage_first: game data in the preparation stage which is in the form of a string; record_step: current step count

```

1: Set Stage_first = "stage[1]:";
2: for i = 1; i < 14; i++ do
3:   for j = 1; j < 14; j++ do
4:     if Phase_last[i, j] == Phase_now[i, j] then
5:       Continue;
6:     else
7:       if Phase_now[i, j] == 1 then
8:         Stage_first += "B (chr(j + 65), chr(4-i))";
9:       else
10:        Stage_first += "W (chr(j + 65), chr(4-i))";
11:      end if
12:    end if
13:  end for
14: end for
15: record_step += 1;
16: return Stage_first and record_step

```

final positions of the player who eats the pieces. Finally, if only one piece is captured, it is determined whether to skipping to eat or eating seeds in a square and record the path. If the value of count is not 1, use Breadth First Search (BFS) to search for the move path, as shown in Algorithm 3.

4 Experiment and Analysis

4.1 Datasets

The experimental datasets are derived from the video of the Jiu Chess National Competition recorded in Xining, Qinghai Province, China in November 2023. In order to improve the accuracy of chess piece recognition, the data enhancement processing of the original image is required. The Laplace sharpening algorithm is used to sharpen the acquired images, as shown in Fig. 7a. Then, the sharpened image is converted to grayscale, and Gaussian filtering is used to obtain the corrected original image, as shown in Fig. 7b. Then, the Canny operator is used for edge detection processing, and Fig. 7c is obtained. Finally, the largest convex quadrilateral is found, the chessboard area is obtained, the four corners of the chessboard are found, calibrated, and read, by the

Algorithm 3 Jiu chess game conversion of battle stage

Require: Phase_last, Phase_now, and record_step

Ensure: Stage_second: chess game data in string form, record_step: number of moves in current step count

```

1: Set Stage_second = "Stage [2]:", string = "", initial = [ ],
   final = [ ], count = 0, Queue = [ ], and data = [ ];
2: for i from 1 to length (Phase_last) do
3:   for j from 1 to length (Phase_last) do
4:     if Phase_last[i, j] == Phase_now[i, j] then
5:       Continue;
6:     else
7:       if Phase_last[i, j] == 2 and Phase_now[i, j] == 0
         then
8:         string ← find_piece_captured; //Capture the
           changes of the piece
9:         count ← count + 1; //count is the current number of
           pieces
10:       else
11:         find_piece_capture;
12:       end if
13:     end if
14:   end for
15: end for
16: if count == 1 then
17:   Determine whether to catch in an FC or TC;
18: else
19:   Search for a piece path by using BFS;
20: end if
21: record_step += 1;
22: return Stage_second and record_step

```

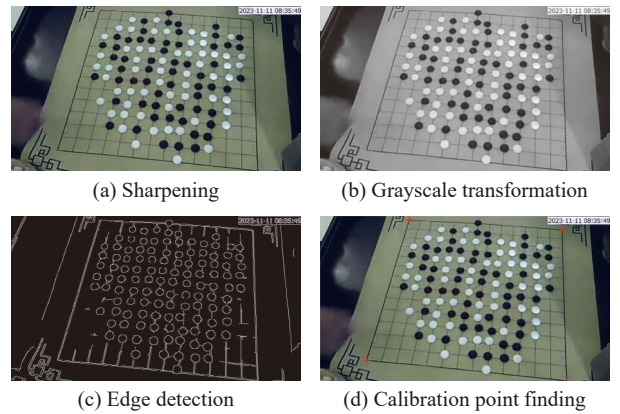


Fig. 7 Raw image data enhancement.

threshold segmentation method, and the positions corresponding to the calibration points are shown in Fig. 7d.

The area in the middle of the calibration point is extracted, as shown in Fig. 8a. Set the calibration point

be Q_1 , Q_2 , Q_3 , and Q_4 , forming an irregular quadrilateral, which is transformed into Fig. 8b by perspective transformation, and the principle of perspective transformation is shown in Fig. 9. Observing Fig. 8b, it can be found that the chess pieces on the outer grid of the chessboard will be partially missing, which will directly affect the later detection of chess pieces. To solve this problem, this article adjusts the expected chessboard grid positions inward. Specifically, when the size of the standard image is set to 660 pixel $\times$ 660 pixel, with the upper left corner as the origin, the coordinates of the four corners are (0, 0), (0, 660), (660, 0), (660, 660), and (660, 660) clockwise. The long and wide Jiu chessboard has 14 lines each, with a total of 169 chessboard squares and 196 intersections. Since 660 is not divisible by 14, the length and width are divided into 15 parts according to the characteristics of the board of Jiu chess, of which 2 parts are 18 pixel, and the rest are 48 pixel, that is, the size of a chessboard grid is 48 pixel $\times$ 48 pixel, and 18 pixel is the position of the periphery of the reserved chessboard grid. The coordinates of the four vertices of the original image plane change from (0, 0), (0, 660), (660, 0), (660, 660) to (18, 18), (18, 642), (642, 18), and (642, 642), respectively as shown in Fig. 8c, so that the standard image obtained by the perspective transformation will retain the data on the periphery of

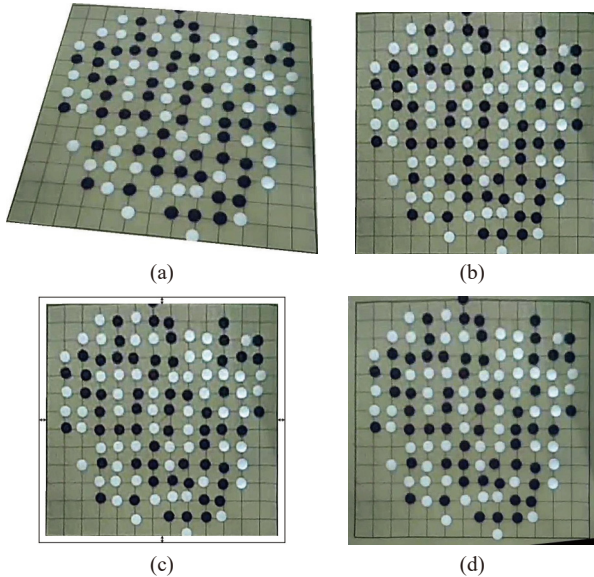


Fig. 8 Process of converting the extracted chessboard into a standard image, (a) is the area of the checkerboard extraction, (b) is the result of the perspective transformation of (a) directly, (c) is the method of adjusting the checkerboard lines inward, and (d) is a standard image.

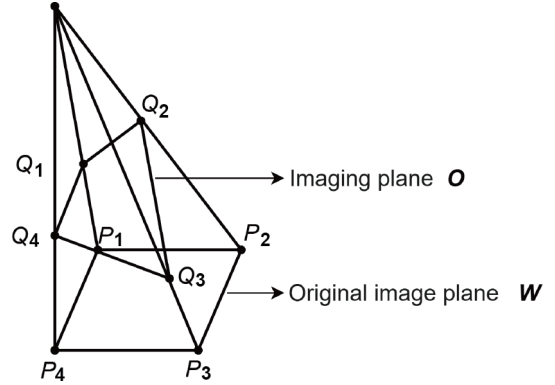


Fig. 9 Principle of perspective transformations.

the chessboard grid, as shown in Fig. 8d.

It is known that the plane where the quadrilaterals $Q_1Q_2Q_3Q_4$ is located is the imaging plane O , and there is a square W on the original image plane $P_1P_2P_3P_4$, and if there is a point Q in the quadrilateral, the coordinates of the Q point are defined as

$$Q = (u, v, 1) \quad (4)$$

where u and v are the coordinate values of the imaging plane O where the point Q is located. Since the image is a two-dimensional plane, the last value in the 3D spatial relationship parameter is set to 1 by default. Let there be a point P' in the square $P_1P_2P_3P_4$, and define the coordinates of the point P as

$$P' = (x', y', z') \quad (5)$$

where x' and y' are the coordinate values of the original image plane where the point P' is located, and z' is the relationship parameter of the three-dimensional space.

The transformation relationship between the points P' and Q is

$$\begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \quad (6)$$

where a_{ij} is the value of the perspective transformation matrix.

Expanded Eq. (6) is as follows:

$$x' = a_{11}u + a_{12}v + a_{13} \quad (7)$$

$$y' = a_{21}u + a_{22}v + a_{23} \quad (8)$$

$$z' = a_{31}u + a_{32}v + a_{33} \quad (9)$$

Substituting the vertex coordinates of Q_1 , Q_2 , Q_3 , and Q_4 of the quadrilateral and the vertex coordinates of P_1 , P_2 , P_3 , and P_4 of the square into Eqs. (7)–(9) to find the perspective transformation matrix,

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \quad (10)$$

The processed dataset contains a total of 2 detection targets, black and white chess pieces. The training set, validation set, and test set consist of 80, 12, and 23 images, respectively.

4.2 Experimental environment setting and evaluation indicators

In this study, the Python programming language is used, the open-source deep learning framework PyTorch is used as the network framework, and CUDA 11.7 is used for accelerated training. The model's hardware test environment includes an Intel(R) Core(TM) i5-12500H CPU and an NVIDIA RTX 3050 with 16 GB of memory. During training, the input image is set to 640 pixel $\times$ 640 pixel, and Stochastic Gradient Descent (SGD) is used as the optimization function for model training. The model training epochs are set to 300, the batch size is 16, and the initial learning rate is 0.01.

In this paper, the evaluation indexes include precision (P), recall (R), F_1 -score (F_1), and mean average accuracy (mAP). Among them, the accuracy and recall rate are used as the basic indicators, and the F_1 score and mAP are calculated as the final evaluation indicators to measure the recognition accuracy of the model. Latency and Framework Per Second (FPS) are used to measure the speed of model detection. The formula for calculating each evaluation index is as follows:

$$P = \frac{TP}{TP + FP} \times 100\% \quad (11)$$

$$R = \frac{TP}{TP + FN} \times 100\% \quad (12)$$

$$F_1 = \frac{2 \times P \times R}{P + R} \times 100\% \quad (13)$$

$$AP = \int_0^1 P(R) dR \quad (14)$$

$$mAP = \frac{1}{n} \sum_{i=1}^n AP_i \quad (15)$$

$$\text{Latency} = \frac{\text{FLOPs}}{\text{FLOPS}} = \frac{1}{\text{FPS}} \quad (16)$$

In the above formulas, the meaning of each variable are as follows.

True Positive (TP) means that the algorithm has

actually detected the number of pieces, False Positive (FP) means that the algorithm has incorrectly marked the area where there are no pieces as pieces, and False Negative (FN) means that the algorithm has failed to accurately detect the actual number of pieces. The value of n is 2, which indicates a black and white piece. mAP is the average of black and white pieces of accuracy. The precision of each category is expressed by its corresponding Average Precision (AP). The mAP50 represents the value of the mAP calculated at the 50% confidence threshold, mAP50-95 is the mAP value calculated between 50% and 95% confidence threshold, and the mAP50-95 can be used to evaluate the robustness and performance of the model well. FLOPs and FLOPS refer to the number of floating-point operations per second and the number of floating-point operations, respectively, and can increase the computation speed by reducing FLOPs and increasing FLOPS. FPS is an important evaluation index to measure the detection speed of the algorithm, and a higher FPS value indicates faster processing speed.

4.3 Experiment results and analysis

Bounding box regression loss (box_loss) optimizes geometric matching between predicted and ground-truth bounding boxes via the Complete Intersection over Union (CIoU) loss, effectively addressing positional deviations and shape mismatches. Classification loss (denoted as cls_loss) enhances classification robustness and mitigates overfitting through continuous label cross-entropy. Distribution focal loss (denoted as df_loss) refines continuous value prediction distributions, improving regression task accuracy. Together, these components form YOLOv8n integrated loss function framework, striking an optimal balance among inference speed, detection precision, and model generalizability across diverse scenarios. cls_loss is calculated as follows:

$$L_{cls} = \frac{1}{N} \sum_{i=1}^N \left[-\left(s'_i \cdot \log(p_i) + (1 - s'_i) \cdot \log(1 - p_i) \right) \right] \quad (17)$$

where N is the number of samples in a training batch, p_i is the predicted probability of the i -th sample belonging to the target class, s'_i is tag integral value and $\log(\cdot)$ is natural logarithm operator, penalizing deviations between predictions and labels.

CIoU loss function works the same way as YOLOv5 does. dfl_loss function is described as follows:

$$\text{DFL}(s_i, s_{i+1}) = (s'_{i+1} - s') \cdot \log(s_i) + (s' - s'_i) \cdot \log(s_{i+1}) \quad (18)$$

where s_i and s_{i+1} are the distribution scores for adjacent discretized intervals of a target property, and s'_i and s'_{i+1} are discretized adjacent tag integral values bracketing the true value s' .

YOLOv8n-ConvNextV2-GDh is used for model training, and the initial epoch of model training is set to 300. As shown in Fig. 10, with the increase of the number of iterations, the box_loss, cls_loss, and dfl_loss all decrease continuously and finally tend to be stable when the epoch is 210. The average processing speed of an image is 13.8 ms, and the total training time of the model is 64.4 min.

4.3.1 Ablation experiments and analysis

In order to investigate whether the improved module of YOLOv8n-ConvNextV2-GDh is effective, this paper uses the self-built dataset and based on the YOLOv8n model to carry out the Ablation Experiment of the ConvNext V2 Module, the GD Mechanism of Adaptive YOLOn, the Optimized Detect Module, and the Channel Pruning, respectively. The results are shown in Table 2.

By replacing the original head network with the

ConvNext V2 module, the model achieves a balanced improvement in both detection accuracy and inference speed. Specifically, P , R , F_1 and mAP50 metrics increase by 0.034%, 0.009%, 0.022%, and 0.002%, respectively. Meanwhile, the inference latency is significantly reduced to 18 ms. This demonstrates that the ConvNextV2 module effectively streamlines computational operations in the detection head while preserving feature representation capacity. Although the accuracy gains are modest in absolute terms, the concurrent reduction in latency highlights its practical value for real-time applications, where maintaining detection performance with faster response times is critical. The GD mechanism is introduced into the neck structure, which slightly improves certain measurement accuracy metrics. However, the corresponding detection speed improvement is limited to 0.8 ms, falling short of the real-time requirements for chess piece detection tasks. Subsequent optimizations targeting both the head and neck network architectures yield consistent enhancements: all accuracy evaluation metrics (P , R , F_1 , mAP50, and mAP50-95) show incremental gains compared to the baseline model, while detection speed improves by 7.5 ms. As shown in

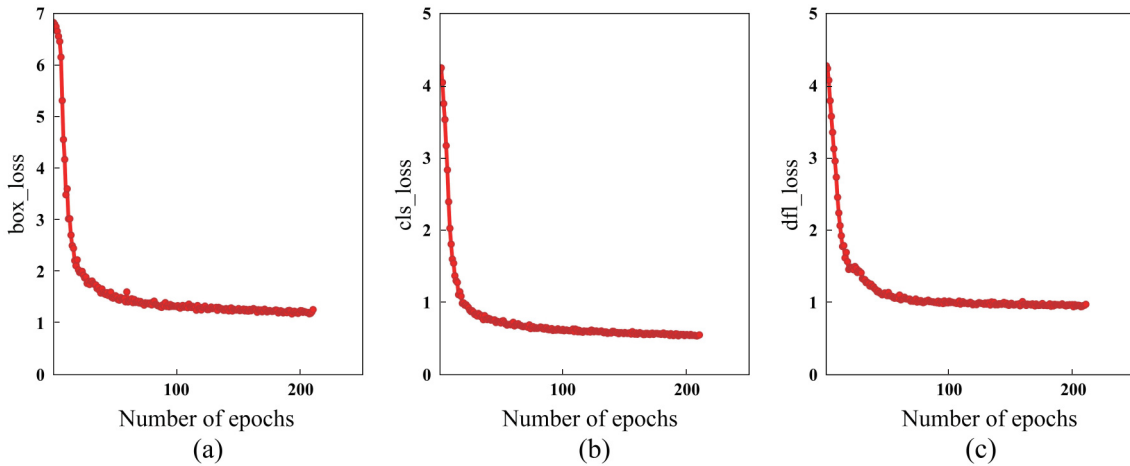


Fig. 10 Curve of the loss function with epoch.

Table 2 Results of ablation experiment. The symbol “×” indicates that the module is not included in the experimental model, while “√” indicates that the module is included in the model. The boldface entries denote the experimental results obtained from our proposed model.

ConvNextV2	GD	Optimized detection	YOLOv8n	P (%)	R (%)	F_1 (%)	mAP50 (%)	mAP50-95 (%)	Latency (ms)
×	×	×	√	99.636	99.938	99.786	99.339	66.526	28.4
√	×	×	√	99.670	99.947	99.808	99.341	66.526	18.0
×	√	×	√	99.652	99.927	99.789	99.344	66.199	27.6
√	√	×	√	99.690	99.939	99.814	99.384	67.583	20.9
√	√	√	√	99.660	99.938	99.799	99.378	66.649	13.8

Table 2. Further refinement of the detect module within the head structure achieves faster inference speeds (13.8 ms), while maintaining accuracy levels surpassing those of YOLOv8n. Finally, the channel pruning module reduces the average processing time of an image to 8.3 ms, which is 20.1 ms faster than that of the YOLOv8n model. The channel pruning module reduces the average processing time of an image to 8.3 ms, which is 20.1 ms faster than that of the YOLOv8n model. It can be seen that although the accuracy of the pruned model is lost to a certain extent, the loss is less. The improved algorithm can greatly improve the speed of chess piece detection under the premise of small loss of accuracy.

4.3.2 Comparative experiments and analysis

The effectiveness of YOLOv8n-ConvNextV2-GDh is confirmed by ablation experiments, and the experimental results are also compared with other mainstream object detection algorithms, Faster R-CNN, YOLOv5, YOLOv7-tiny, YOLOv7, YOLOv10^[35], YOLOv8n, YOLOv8n-FastNet, YOLOv8n-MobileNet, and Gold-YOLO, as shown in Table 3. When the YOLOv8n model is used for chess pieces detection, the values of each accuracy and stability index are higher than those of the Faster R-CNN, YOLOv5, YOLOv7-tiny, YOLOv7, YOLOv10, YOLOv8n-FastNet, and YOLOv8n-MobileNet, and the indexes of P , R , F_1 , mAP50, and mAP50-95 are 99.636%, 99.938%, 99.786%, 99.339%, and 66.526%, respectively. Compared with the Gold-YOLO, the latency value of the YOLOv8n is 28.4 ms, which is 1.2 ms lower than that of the Gold-YOLO model, and the speed of the model is relatively high. Therefore, this paper improves the YOLOv8n model and achieves a good balance between the speed and accuracy of the

model based on the task requirements of the actual Jiu chess piece detection. In contrast, compared with the YOLOv8n model, the processing time of the our model is shortened to 13.8 ms, and the values of P , F_1 , mAP50, and mAP50-95 are increased by 0.024%, 0.013%, 0.039%, and 0.123%, respectively, and the model achieves a good balance between accuracy and speed.

In addition, Figs. 11a and 11b show the curve change plots of mAP50 and mAP50-95 (training stage) of YOLOv8n series algorithms, respectively, the value of mAP can better evaluate the robustness and performance of the model. As can be seen from Fig. 8, the red curve is our model, which has the fastest convergence speed and the highest accuracy compared with other models. In this paper, there are outliers in the early stage of training, but with the increase of the number of iterations, the convergence of the model gradually tends to be stable, and it is found that the oscillation of the model in the early stage does not affect the performance of the model.

4.3.3 Model pruning experiments and analysis

The optimum pruning ratio is discovered through extensive experimental exploration. The LAMP channel pruning algorithm is used to prune the improved YOLOv8n model with different amplitudes. Then, different models after pruning are trained, and the experimental results of different models are compared. The accuracy, speed, and stability of the model are analyzed to derive the optimum pruning ratio.

The experimental methods to explore the optimal pruning rate are as follows. In the experiment, the pruning ratio is controlled by a parameter named speed_up, where a value of 2.0 indicates that the

Table 3 Results of the comparative experiments. The boldface entries denote the experimental results obtained from our proposed model.

Method	P (%)	R (%)	F_1 (%)	mAP50 (%)	mAP50-95 (%)	Latency (ms)
Faster R-CNN	64.350	33.765	44.290	35.310	37.432	125.2
YOLOv5	96.560	99.450	97.983	99.127	63.737	32.1
YOLOv7-tiny	96.842	99.325	98.068	99.230	61.565	29.3
YOLOv7	99.630	99.921	99.775	99.013	64.470	23.1
YOLOv10	97.178	96.214	96.693	99.059	65.980	5.84
YOLOv8n	99.636	99.938	99.786	99.339	66.526	28.4
YOLOv8n-FastNet	99.628	99.938	99.783	99.301	66.440	18.6
YOLOv8n-MobileNet	99.626	99.911	99.768	99.154	66.496	19.7
Gold-YOLO	99.652	99.927	99.799	99.344	66.199	29.6
Ours	99.660	99.938	99.799	99.378	66.649	13.8

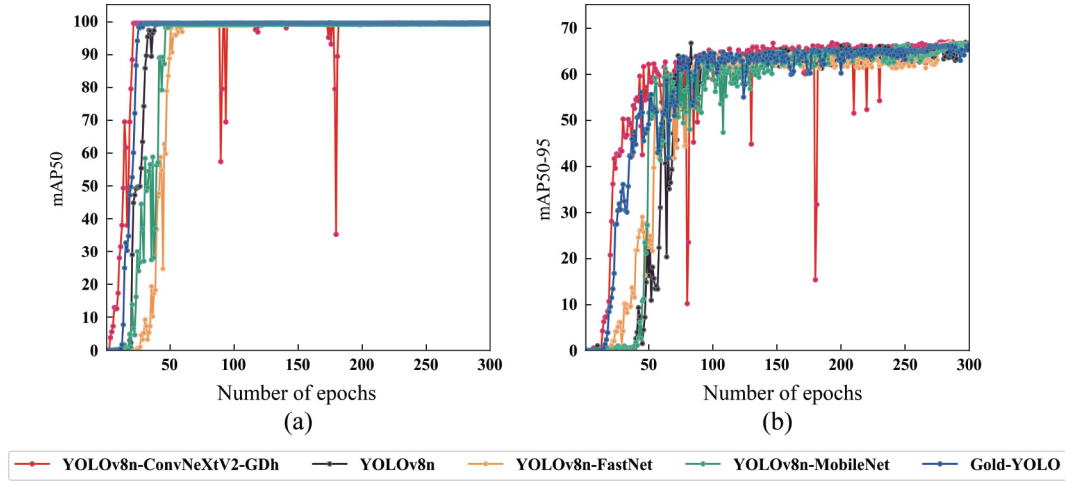


Fig. 11 Comparison of YOLOv8n-ConvNeXtV2-GDh.

pruned model retains half of the number of parameters compared to its original counterpart. A total of 11 sets of comparison experiments exp1–exp11 are carried out by setting the initial value of speed-up to 2.0, the termination value to 3.0, and the intermediate step size to 0.1. The experimental results are shown in Table 4. FPS, P , mAP50, and mAP50-95 are selected as the evaluation indexes of speed, accuracy, and stability of the model, and the best trimming rate is obtained by comparing the experimental results. The following is an explanation of the default value setting of the speed_up to 2.0. When the improved YOLOv8n model is not pruned, it occupies about 16.9 Megabit of memory. Experiments show that when the speed_up value is 2.0, the memory occupied by the model is 8.9 Megabit, and pruning reduces the memory space

occupied by the model to the extent that it is basically consistent with the memory space occupied by YOLOv8n. Further increasing the pruning ratio can not only reduce the memory occupation of the model to a smaller footprint than YOLOv8n, but also be more conducive to exploring the performance of other parameters of the model. The following is explained that the termination value of the speed_up is set to 3.0. When the speed_up value is 3.0, the accuracy and speed of the model are reduced, and more importantly, the stability of the model is low, so the maximum amplitude of pruning is set to 3.0. The following is an explanation of how the step size of the experiment is set to 0.1. Experiments show that when the step size is set to 0.1, when the model is set to different speed_up values, several models occupy the same memory space.

Table 4 Model training results under different pruning amplitudes.

Experiment	speed_up	Memory size (Megabit)	FPS (Hz)	P (%)	R (%)	F_1 (%)	mAP50 (%)	mAP50-95 (%)
Baseline	–	16.9	7.2	99.660	99.938	99.799	99.378	66.649
exp1	2.0	8.9*	94.3	99.639	1.000	99.819	99.377	66.600
exp2	2.1	8.7	94.6	99.638	1.000	99.819	99.376	66.643
exp3	2.2	8.4	96.5	99.637	1.000	99.818	99.360	66.641
exp4	2.3	8.2	104.0	99.636	1.000	99.818	99.360	66.640
exp5	2.4	7.9	110.3	99.635	1.000	99.817	99.357	66.638
exp6	2.5	7.8	114.3	99.635	1.000	99.817	99.345	66.628
exp7	2.6	7.6	114.0	99.633	99.939	99.786	99.342	66.625
exp8	2.7	7.5	120.3	99.629	99.939	99.784	99.336	66.604
exp9	2.8	7.5	117.2	99.626	1.000	99.813	99.332	65.734
exp10	2.9	7.3	130.7	99.677	1.000	99.838	99.310	65.704
exp11	3.0	7.3	106.2	99.669	1.000	99.834	99.310	65.484

Note: The memory size of exp1 is 8.9 Megabit, which is the same size as YOLOv8n. The results of each evaluation index before the pruning experiment are shown in the first row in bold, and a set of experimental results on the optimal level of pruning explored in this paper are shown in the ninth row in bold.

When the step size is set to a different value (e.g., 0.05), it is found that when setting different speed_up values, the number of models with the same memory space is more, it takes more time and computing resources to perform comparative experiments, and these models have less variation in accuracy, speed, and stability. Therefore, considering the balance between resource consumption and experimental results, it is reasonable to set the pruning step size to 0.1.

A good balance between speed, accuracy, and stability is the criterion for the best pruning rate. FPS, P , mAP50, and mAP50-95 are selected as indicators for speed, accuracy, and stability, as shown in Fig. 12. As can be seen from Fig. 12, when the value of the speed_up increases from 2.6 to 2.7, the accuracy of the model decreases steadily according to the original trend, the speed increases steadily, the change values of mAP50 and mAP50-95 are relatively small, and when the speed_up changes from 2.7 to 2.8, the stability of the model decreases sharply, and the values of P and FPS decrease steadily. Therefore, when the value of the

speed_up is 2.7, the training effect of the model is relatively the best.

4.3.4 Visualization of chess piece recognition results

Figures 13a–13c represent images of the preparation phase, and Figs. 13d–13f represent images of the combat phase. Figures 13a and 13d are the standard images obtained after the perspective transformation, which are used as the original input images for the chess piece detection experiment. The original YOLOv8n model is used for chess piece detection, and the visualization results are shown in Figs. 13b and 13e, where the red box is the object detection frame of the black chess piece, the blue box is the object detection frame of the white chess piece, and the number above the detection box represents the accuracy of recognition. The improved YOLOv8n model is used for chess piece detection, and the visualization results are shown in Figs. 13c and 13f. By comparing Figs. 13b and 13c or Figs. 13d and 13e, it is verified that the detection speed of the improved YOLOv8n model is reduced compared with the

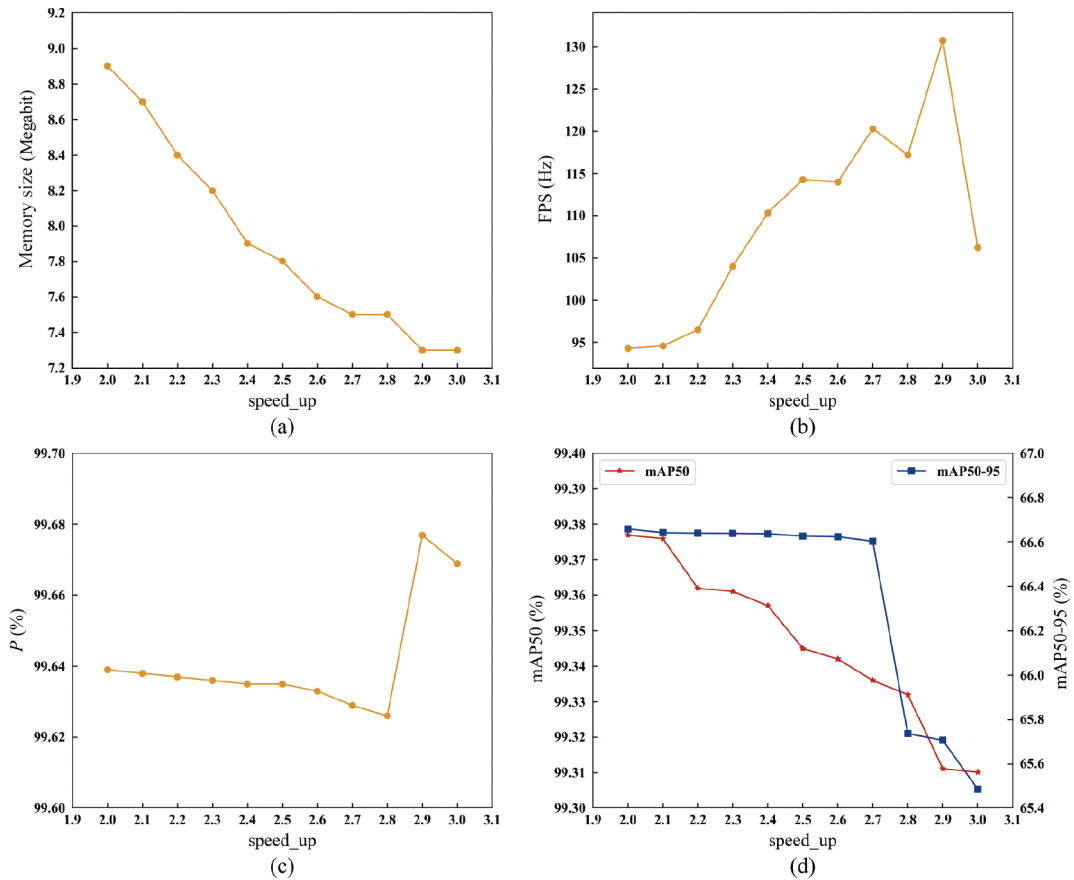


Fig. 12 Results of pruning evaluation indexes for different speed_up channels.

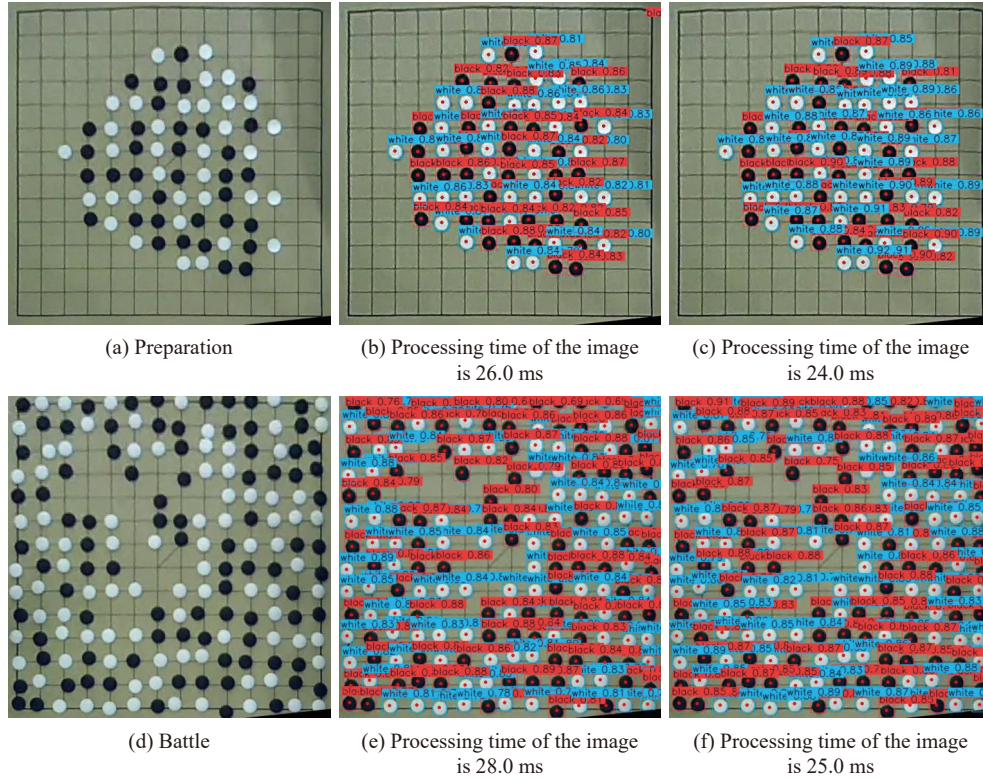


Fig. 13 Visualization of the chess pieces recognition results, (a) is a standard image of the preparation stage, (b) is the result of YOLOv8n's recognition of (a), (c) is the result of our model's recognition of (a), (d) is a standard image of the battle stage, (e) is the result of YOLOv8n's recognition of (d), and (f) is the results of our model's recognition of (d).

YOLOv8n model, and the accuracy of the detection of chess pieces in the same position is found to be higher than that of the YOLOv8n-ConvNextV2-GDh model. For example, the values of accuracy of the three white pieces on the far right in Fig. 13b are 83%, 81%, and 80% from top to bottom, while the values of accuracy of the three white pieces on the far right in Fig. 13c are 86%, 89%, and 89% from top to bottom.

4.4 Discussion

The YOLOv8n-ConvNextV2-GDh model proposed in this paper adopts an end-to-end prediction method, which has the advantages of high detection accuracy, fast processing speed, and easy deployment, and can detect Jiu chess pieces with high speed and high precision. However, the experimental dataset in this paper is small and the external interference factors in the actual collection are not excluded, so the quality of the dataset needs to be further improved. At this stage, the realization of intelligent collection tasks still needs to be carried out step by step, first of all, it is necessary to manually collect the video data of the game, and then process the video images to complete the chess

score collection task. The ultimate goal of this study is to realize real-time game conversion, and it is the most challenging task to eliminate external interference factors in actual game and realize the automatic judgment of chess players' chess speed, and the research in this paper needs to be further expanded and deepened.

5 Conclusion

High-quality Tibetan Jiu chess game data are important basis for Jiu chess research, but the long-term problem for Jiu chess researchers is that the manual collection of high-quality chess manual data is time-consuming and labor-intensive. In order to solve the dilemma of high-quality chess manual data collection, this paper proposes an intelligent data collection algorithm for Jiu chess game based on the improved YOLOv8n, which improves the YOLOv8n model, and the accuracy of the improved model's chess pieces detection reaches 99.660%, and the speed of processing a picture is only 13.8 ms, which is 14.6 ms faster than that of the YOLOv8n model, which can achieve high-speed and high-precision detection of chess pieces in the process

of Jiu chess matching. The proposed LAMP score channel pruning algorithm achieves a substantial reduction in parameter scale, decreasing the model size to 35.7% of its original configuration, while significantly accelerating inference speed, achieving an image processing time of 8.3 ms. The proposed nearest Euclidean distance information matching algorithm and the game conversion algorithm in the preparation stage and battle stage finally convert the results of model recognition into standard Jiu chess manual data.

In the future, we will continue to improve the accuracy of chess manual data collection. we realize the automatic judgment of the chess player's chess speed to find the best interception time of the video frame image, save computing resources, and prevent the problem of missing steps and missing pieces as much as possible. Furthermore, the external interference factors of image collection, such as the interference of lighting conditions and the limited hardware conditions, under real conditions, the lighting conditions of different venues, are different, which is directly related to the accuracy of chess piece recognition. The problem of hardware equipment, such as the venue and video recording, is a force majeure factor in the research process.

Acknowledgment

This work was supported by the National Natural Science Foundation of China (Nos. 62276285 and 62236011) and the Major Projects of Social Science of China (No. 20&ZD279)

References

- [1] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al., Human-level control through deep reinforcement learning, *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [2] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, et al., Mastering the game of go with deep neural networks and tree search, *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.
- [3] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, et al., Mastering the game of go without human knowledge, *Nature*, vol. 550, no. 7676, pp. 354–359, 2017.
- [4] N. Brown and T. Sandholm, Superhuman AI for heads-up no-limit poker: Libratus beats top professionals, *Science*, vol. 359, no. 6374, pp. 418–424, 2017.
- [5] N. Brown and T. Sandholm, Superhuman AI for multiplayer poker, *Science*, vol. 365, no. 6456, pp. 885–890, 2019.
- [6] H. Fu, W. Liu, S. Wu, Y. Wang, T. Yang, K. Li, J. Xing, B. Li, B. Ma, Q. Fu, et al., Actor-critic policy optimization in a large-scale imperfect-information game, in *Proc. 10th Int. Conf. Learning Representations*, Virtual Event, <https://openreview.net/forum?id=DTXZqTnV5nW>, 2021.
- [7] X. Li, S. Wang, Z. Lv, Y. Li, and L. Wu, Strategy research based on chess shapes for Tibetan JIU computer game, *ICGA J.*, vol. 40, no. 3, pp. 318–328, 2018.
- [8] X. Li, Z. Lv, L. Wu, Y. Zhao, and X. Xu, Hybrid online and offline reinforcement learning for Tibetan Jiu chess, *Complexity*, vol. 2020, p. 4708075, 2020.
- [9] X. Li, Y. Chen, Y. Zhang, B. Liu, and L. Wu, A phased game algorithm combining deep reinforcement learning and UCT for Tibetan Jiu chess, in *Proc. IEEE 47th Annu. Computers, Software, and Applications Conf. (COMPSAC)*, Torino, Italy, 2023, pp. 390–395.
- [10] Y. Chen, L. Wu, J. Yan, Y. Zhang, X. Li, and X. Yang, Tibetan Jiu chess intelligent game platform, in *Cognitive Computation and Systems*, F. Sun and J. Li, eds. Singapore: Springer, 2024, pp. 322–333.
- [11] X. Zhao and X. Zhao, Weiqi recognition method under uneven illumination based on neural networks, (in Chinese), *Software Eng.*, vol. 25, no. 11, pp. 1–4, 2022.
- [12] J. Song and S. Li, A robust algorithm for go image recognition in go game, in *Proc. IEEE 4th Int. Conf. Computer and Communications (ICCC)*, Chengdu, China, 2018, pp. 1397–1401.
- [13] Q. Zhang, M. Zhang, T. Chen, Z. Sun, Y. Ma, and B. Yu, Recent advances in convolutional neural network acceleration, *Neurocomputing*, vol. 323, pp. 37–51, 2019.
- [14] K. Simonyan and A. Zisserman, Very deep convolutional networks for large-scale image recognition, arXiv preprint arXiv: 1409.1556, 2014.
- [15] K. He, X. Zhang, S. Ren, and J. Sun, Deep residual learning for image recognition, in *Proc. 2016 IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778.
- [16] K. He, X. Zhang, S. Ren, and J. Sun, Identity mappings in deep residual networks, in *Proc. 14th European Conf. Computer Vision*, Amsterdam, the Netherlands, 2016, pp. 630–645.
- [17] E. Shelhamer, J. Long, and T. Darrell, Fully convolutional networks for semantic segmentation, *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 4, pp. 640–651, 2017.
- [18] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, Densely connected convolutional networks, in *Proc. 2017 IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Honolulu, HI, USA, 2017, pp. 2261–2269.
- [19] J. Terven and D. Córdova-Esparza, A comprehensive review of YOLO: From YOLOv1 to YOLOv8 and beyond, arXiv preprint arXiv: 2304.00501, 2023.
- [20] Z. Zou, K. Chen, Z. Shi, Y. Guo, and J. Ye, Object

- detection in 20 years: A survey, *Proc. IEEE*, vol. 111, no. 3, pp. 257–276, 2023.
- [21] R. Girshick, J. Donahue, T. Darrell, and J. Malik, Rich feature hierarchies for accurate object detection and semantic segmentation, in *Proc. 2014 IEEE Conf. Computer Vision and Pattern Recognition*, Columbus, OH, USA, 2014, pp. 580–587.
- [22] R. Girshick, J. Donahue, T. Darrell, and J. Malik, Region-based convolutional networks for accurate object detection and segmentation, *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 38, no. 1, pp. 142–158, 2016.
- [23] A. Krizhevsky, I. Sutskever, and G. E. Hinton, ImageNet classification with deep convolutional neural networks, *Commun. ACM*, vol. 60, no. 6, pp. 84–90, 2017.
- [24] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, You only look once: Unified, real-time object detection, in *Proc. 2016 IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779–788.
- [25] J. Zeng and H. Zhong, YOLOv8-PD: An improved road damage detection algorithm based on YOLOv8n model, *Sci. Rep.*, vol. 14, no. 1, p. 12052, 2024.
- [26] L. Tan, Y. Liang, J. Xia, H. Wu, and J. Zhu, Detection and diagnosis of small target breast masses based on convolutional neural networks, *Tsinghua Science and Technology*, vol. 29, no. 5, pp. 1524–1539, 2024.
- [27] Y. Liu, Z. Shao, and N. Hoffmann, Global attention mechanism: Retain information to enhance channel-spatial interactions, arXiv preprint arXiv: 2112.05561, 2021.
- [28] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, MobileNets: Efficient convolutional neural networks for mobile vision applications, arXiv preprint arXiv: 1704.04861, 2017.
- [29] X. Wang and J. Liu, Vegetable disease detection using an improved YOLOv8 algorithm in the greenhouse plant environment, *Sci. Rep.*, vol. 14, no. 1, p. 4261, 2024.
- [30] W. Liu, L. Zhou, S. Zhang, N. Luo, and M. Xu, A new high-precision and lightweight detection model for illegal construction objects based on deep learning, *Tsinghua Science and Technology*, vol. 29, no. 4, pp. 1002–1022, 2024.
- [31] S. Woo, S. Debnath, R. Hu, X. Chen, Z. Liu, I. S. Kweon, and S. Xie, ConvNeXt V2: Co-designing and scaling ConvNets with masked autoencoders, in *Proc. 2023 IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, Vancouver, Canada, 2023, pp. 16133–16142.
- [32] C. Wang, W. He, Y. Nie, J. Guo, C. Liu, K. Han, and Y. Wang, Gold-YOLO: Efficient object detector via gather-and-distribute mechanism, in *Proc. 37th Int. Conf. Neural Information Processing Systems*, New Orleans, LA, USA, 2023, p. 2224.
- [33] P. Molchanov, A. Mallya, S. Tyree, I. Frosio, and J. Kautz, Importance estimation for neural network pruning, in *Proc. 2019 IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, Long Beach, CA, USA, 2019, pp. 11256–11264.
- [34] J. Lee, S. Park, S. Mo, S. Ahn, and J. Shin. Layer-adaptive sparsity for the magnitude-based pruning, in *Proc. 9th Int. Conf. Learning Representations*, Virtual Event, <https://openreview.net/forum?id=H6ATjJ0TKdf>, 2021.
- [35] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, YOLOv10: Real-time end-to-end object detection, in *Proc. 38th Int. Conf. Neural Information Processing Systems*, Vancouver, Canada, https://proceedings.neurips.cc/paper_files/paper/2024/hash/c34ddd05eb089991f06f3c5dc36836e0-Abstract-Conference.html, 2024.



Li Xiali received the BEng and MEng degrees in computer science from Xi'an Jiaotong University, China in 2001 and 2004, respectively. She is currently a full professor at Minzu University of China, China. Her main research interests include computer games, computer vision, and embodied AI.



Yuanli Jia received the BEng degree from North China University of Science and Technology, China in 2023. She is currently a master student at Minzu University of China, China. Her research interests mainly include computer vision and computer game.



Junzhi Yu received the PhD degree in control theory and control engineering from Institute of Automation Chinese Academy of Sciences, China in 2003. He is now a Boya Distinguished Professor at Department of Advanced Manufacturing and Robotics, College of Engineering, Peking University, China. His research interests include intelligent robots, mechatronics integration, and computational intelligence. He is a fellow of IEEE.



Feifan He received the BEng degree from South-Central University for Nationalities, China in 2023. He is currently a master student at Minzu University of China, China. His research interests mainly include computer vision and game robots.